

Software Requirements Specification

System: SusuBook — Digital Susu Collection and Accountability System **Version:** 1.0 **Standard:** Structured after IEEE Std 830-1998 [29], adapted to the scope of a 48-hour capstone

1. Introduction

1.1 Purpose

This document specifies the requirements for SusuBook, a web-based system that records daily susu contributions, gives clients an independent view of their own savings, and enables same-day reconciliation of field collections against cash remitted to a branch.

It is written for the examiner assessing the system, and for any developer who subsequently maintains or extends it.

1.2 Scope

SusuBook digitises the **record** of susu collection. It does not move money. Cash continues to pass physically from client to collector; the system records that event, derives its consequences, and makes the record visible to every party with a legitimate interest in it.

In scope: client enrolment, contribution cycles, daily contribution recording with validation, the digital susu card, payout computation with commission, daily remittance declaration and variance detection, client self-service access, role-based authorisation, and an append-only audit trail.

Out of scope: mobile money and bank integration (CO-05), SMS notification (FR-31), offline operation, multi-institution tenancy, and native mobile applications. Appendix A records the reasoning; Section 11 (Maintenance and Future Evolution) carries them forward.

1.3 Definitions

Term	Meaning
Susu	A Ghanaian informal savings practice; here, the collector variant.
Client	A saver who contributes a fixed amount daily to a collector.
Collector	The field officer who visits clients, receives cash and records contributions.
Supervisor	A branch officer who oversees collectors, reconciles remittances and authorises payouts.
Daily rate	The fixed amount a client agrees to contribute each day.
Contribution cycle	A fixed-length savings period, 31 days by default, ending in a payout.
Contribution	One recorded payment by a client, allocated to a specific date in a cycle.
Susu card	The 31-day grid view of a cycle, showing each day as paid, missed or pending. Digital equivalent of the paper card.
Commission	The collector's fee, one day's contribution, retained at payout.
Payout	The net amount released to the client at maturity: total collected less commission.
Remittance declaration	A collector's statement of cash banked at the branch for a given day.
Variance	The difference between contributions recorded in the field and cash declared, for one collector on one day.
Reversal	A linked correction entry that negates an earlier contribution without deleting it.
Pesewa	One hundredth of a Ghana Cedi (GHS). All monetary values are stored as integer pesewas.
Public reference	An opaque, non-sequential UUIDv4 identifying a client in externally visible URLs. Identifies but does not authorise.
QR card	A printed card carrying a client's public reference as a QR code encoding their contribution URL. The digital successor to the paper susu card.

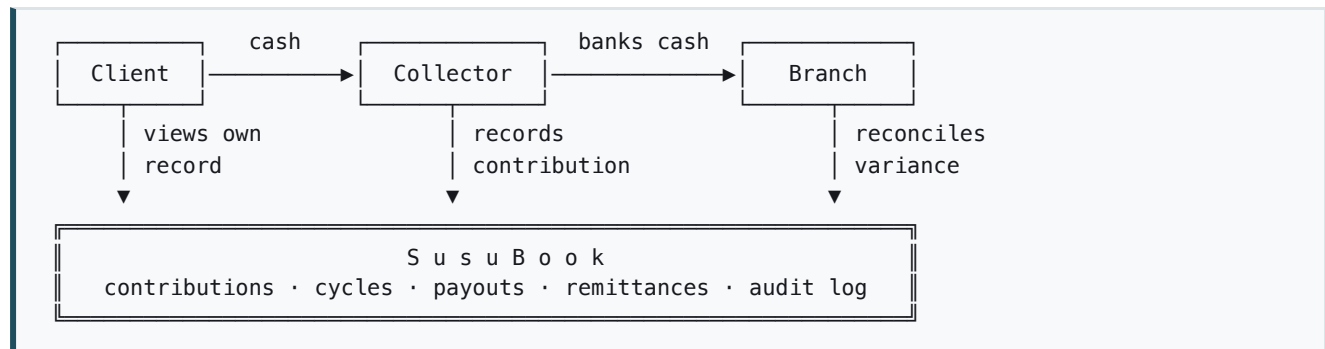
1.4 References

- CSCD602 Session 2 — Requirements Engineering
- CSCD602 Session 3 — Technical Debt
- CSCD602 Session 5 — Software Design and Architecture
- CSCD602 Session 6 — Software Effort Estimation
- Data Protection Act 2012 (Act 843), Republic of Ghana
- Section 1 (Problem Definition), Section 2 (Stakeholder Analysis), Section 3 (Requirements Analysis)

2. Overall description

2.1 Product perspective

SusuBook is a new, self-contained system. It replaces a paper artefact, the susu card, rather than an existing software system, so there is no legacy data migration and no external interface (hence zero External Interface Files in the function point count, Section 4.2.2 (Count A: full specified scope (all 38 functional requirements)) of the estimation).



The system's defining property is that the client's arrow **into** it is independent of the collector's. That is what the paper card cannot provide.

2.2 User classes

Class	Frequency of use	Technical skill	Device
Client	Weekly to monthly	Low; some limited literacy	Low-end Android, mobile data
Collector	Many times daily	Moderate	Low-end Android, mobile data, outdoors
Supervisor	Daily	Moderate	Desktop or tablet at branch
Administrator	Occasional	High	Desktop

2.3 Operating environment

Server-rendered web application over HTTPS, accessed through a mobile browser. Runs on Python 3.12 with Flask, against PostgreSQL 16. Identical database engine in development (Docker) and production (managed Postgres) per NFR-10.

2.4 Design and implementation constraints

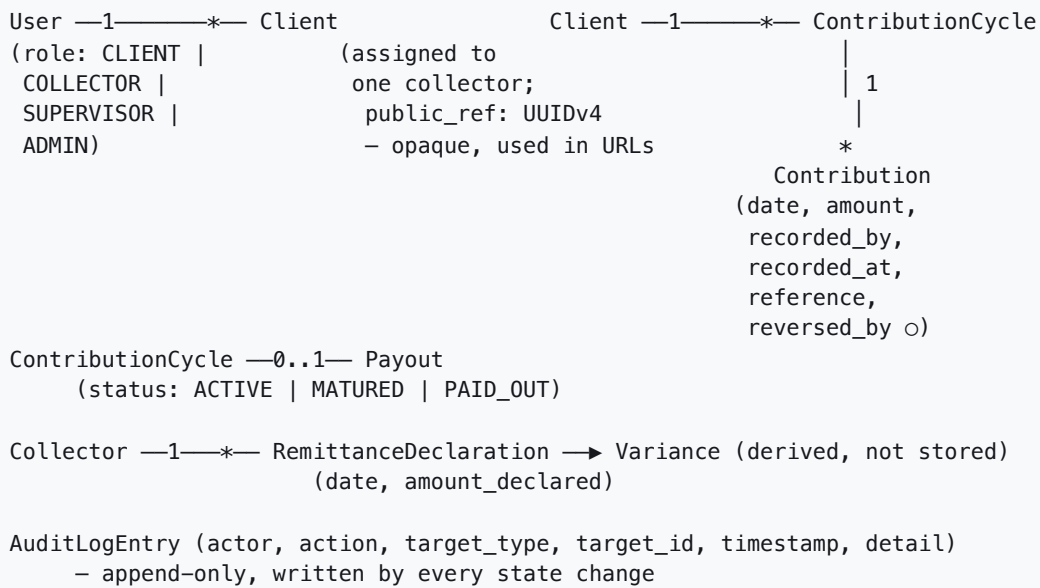
Carried from Section 3.5 (Constraints): 48-hour window (CO-01), single developer (CO-02), public deployment required (CO-03), free-tier hosting (CO-04), no payment API access (CO-05), Act 843 compliance (CO-06), low-end devices on mobile data (CO-07).

2.5 Assumptions and dependencies

Carried from Section 2.5 (Assumptions requiring stakeholder validation), of which **A5 is critical**: clients are assumed able to reach a mobile web page. If false, the system's central value (independent client visibility) requires an SMS channel instead.

3. System features

3.1 Domain model



3.2 Business rules

These are the rules the domain layer enforces, independently of any user interface. They are the primary subject of unit testing.

ID	Rule
BR-R1	All monetary values are integer pesewas. Floating-point arithmetic is never applied to money.
BR-R2	A client has at most one ACTIVE cycle at any time.
BR-R3	A contribution must be allocated to a date within its cycle's start and end dates inclusive.
BR-R4	A contribution date may not be in the future.
BR-R5	At most one non-reversed contribution may exist per client per date.
BR-R6	No contribution may be recorded against a cycle whose status is MATURED or PAID_OUT.
BR-R7	A contribution amount must be a positive whole multiple of the client's daily rate.
BR-R8	Payout = total collected – one day's rate (commission).
BR-R9	Where total collected $\leq$ one day's rate, payout is zero and the whole balance is retained as commission.
BR-R10	A cycle may be paid out at most once.
BR-R11	A contribution is never edited or deleted; correction is a linked reversal entry.
BR-R12	A cycle reaches MATURED when the current date passes its end date, regardless of days paid.
BR-R13	Variance for a collector on a date = sum of contributions recorded – amount declared.
BR-R14	Public references exposed in URLs or QR codes are opaque and non-sequential (UUIDv4). Sequential database identifiers never appear in a URL. (CR-001)
BR-R15	A client reference identifies; it does not authorise. Possession confers no permission, authorisation remains the collector–client assignment enforced server-side (FR-05). (CR-001)

BR-R14 and BR-R15 together are what make a QR card safe to carry through a public market.

The card must be assumed photographable. BR-R14 prevents an observer deriving other clients' references by incrementing a number; BR-R15 ensures that holding a reference grants nothing, because the authorisation decision never consults it. The QR encodes a URL containing an opaque reference and nothing else, no name, phone, balance or rate, so a photographed card leaks no personal data (NFR-06, Act 843).

BR-R9 exists because of a real edge case: a client who contributes on only one day receives nothing, because the single day's contribution *is* the commission. Stating the rule explicitly prevents a negative payout, which is the defect this rule guards against.

3.3 Use cases

ID	Use case	Primary actor	FRs
UC-01	Log in	All	FR-01...04
UC-02	Enrol client	Collector	FR-06, FR-07, FR-10
UC-03	Record daily contribution	Collector	FR-11...14, FR-30, FR-32
UC-04	View digital susu card	Collector, Client	FR-16, FR-17
UC-05	Declare daily remittance	Collector	FR-24
UC-06	Review daily variance	Supervisor	FR-25, FR-26
UC-07	Release matured payout	Supervisor	FR-18...21
UC-08	View own contribution history	Client	FR-28, FR-29
UC-09	Reverse an erroneous contribution	Supervisor	FR-33, FR-32
UC-10	Issue a client QR card (<i>CR-001</i>)	Collector	FR-39

UC-03: Record daily contribution (core use case)

Actor	Collector
Precondition	Collector authenticated; client assigned to this collector; client has an ACTIVE cycle
Postcondition	Contribution persisted with a unique reference; audit entry written; client's view updated
Frequency	Highest in the system, dozens of times per collector per day

Main flow

1. Collector opens today's route sheet.
2. System lists assigned clients with today's collection status.
3. Collector selects a client not yet collected from.
4. System presents a confirmation showing the client's name and their daily rate, pre-filled.
5. Collector confirms.
6. System validates against BR-R3 – BR-R7.
7. System persists the contribution with date, amount, recording collector, server timestamp and a unique reference.
8. System writes an audit entry.
9. System returns the updated route sheet with the client marked collected.

Alternate flows

- **3a. Catch-up payment** (FR-15, *Should*), collector indicates several unpaid days; system allocates the amount across the specific unpaid dates and validates each against BR-R3 and BR-R5.
- **3b. Entry by QR scan** (FR-39, FR-40, *Should (CR-001)*) instead of steps 1–3, the collector scans the client's QR card with the phone's camera application, which opens the client's contribution URL directly. The system resolves the public reference (BR-R14), verifies the client is assigned to this collector (FR-05, BR-R15), and enters the flow at step 4. Reduces the interaction to **scan and confirm**. If the card is missing or unreadable, the collector falls back to the main flow.
- **4a. Amount differs from the daily rate**, collector overrides; system validates BR-R7 (whole multiple of the rate) and rejects any other value.

Exception flows

- **6a. Contribution already exists for this client and date** (BR-R5): system rejects, states the existing reference, records nothing.
- **6b. Date is in the future** (BR-R4): system rejects.
- **6c. Cycle is MATURED or PAID_OUT** (BR-R6): system rejects and directs the collector to the payout process.
- **6d. Client not assigned to this collector** (FR-05): system denies authorisation and records the attempt in the audit log.

Step 4 pre-fills the amount and step 5 is a single confirmation. That is what satisfies NFR-02's three-interaction limit, and it is the design resolution of conflict C1 (collector speed vs. client verifiability) from Section 2.4 (Conflicting stakeholder needs), integrity is added server-side at steps 7 and 8, costing the collector nothing.

UC-07: Release matured payout

Actor	Supervisor
Precondition	Cycle status is MATURED; supervisor authenticated
Postcondition	Payout recorded; cycle set PAID_OUT; a new ACTIVE cycle opened; audit entry written

Main flow

1. Supervisor opens the matured cycles list.
2. System shows each client, days paid, total collected, commission and net payout.
3. Supervisor selects a cycle and confirms release.
4. System re-validates BR-R8 – BR-R10 at the moment of release.
5. System records the payout, sets the cycle PAID_OUT, opens the next ACTIVE cycle.
6. System writes an audit entry and displays a payout advice.

Exception flows

- **4a. Cycle already PAID_OUT** (BR-R10): rejected; no second payout is possible.
- **4b. Total collected $\leq$ daily rate** (BR-R9): payout of zero is shown explicitly with the reason, and requires confirmation rather than failing silently.
- **4c. Cycle not yet MATURED**, rejected; directed to early withdrawal (FR-22) instead.

Remaining use cases in brief

- **UC-01 Log in**, credentials verified against a salted hash; role determines the landing page; failed attempts audited.
- **UC-02 Enrol client**, collector captures name, phone, business type, location and daily rate; system creates the client, assigns them to this collector and opens their first cycle atomically.
- **UC-04 View susu card** — 31-day grid; each day rendered paid, missed or pending; header shows days paid, total collected and projected net payout.
- **UC-05 Declare remittance**, collector enters cash banked for the day; system stores it and computes the variance.
- **UC-06 Review variance**, supervisor sees all non-zero variances for the day, by collector, with the underlying contributions.
- **UC-08 Client history**, client sees every contribution against them with amount, date, time recorded, recording collector and reference; the independent record that answers P1.
- **UC-09 Reverse contribution**, supervisor records a reversal; original remains visible, linked to the reversal; both appear in the audit trail (BR-R11).
- **UC-10 Issue QR card**, collector opens a client's printable card page; system renders the client's public reference as a QR code encoding their contribution URL, with the client's name for human identification; collector prints and issues it. Re-issuing a lost card by rotating the reference is deferred to the evolution plan.

3.4 User stories

As a...	I want to...	So that...	FR
Client	see every payment recorded against me	I can prove what I have saved	FR-28
Client	see what I will receive at maturity	I can plan without asking the collector	FR-29
Collector	record a contribution in a couple of taps	I can move through my route quickly	FR-11, NFR-02
Collector	see who I have not yet collected from today	I do not miss a client	FR-23
Supervisor	see today's variance today	I can act while the money is still recoverable	FR-25, FR-26
Supervisor	authorise payouts	funds are not released without oversight	FR-20
Administrator	restrict a collector to their own clients	client data is not exposed across routes	FR-05

4. External interface requirements

User interfaces. Mobile-first, server-rendered HTML. Minimum 44×44 px touch targets, WCAG 2.1 AA contrast, legible in outdoor light (NFR-08). Four role-specific landing pages.

Hardware interfaces. None. Browser only; no card readers, printers or peripherals.

Software interfaces. PostgreSQL 16 over the SQLAlchemy ORM. No third-party APIs (CO-05).

Communications interfaces. HTTPS only. Session cookies marked `Secure`, `HttpOnly` and `SameSite`. CSRF tokens on every state-changing form.

5. Non-functional requirements

Specified in full at Section 3.4 (Non-functional requirements (how well it must perform)): NFR-01 performance, NFR-02 usability, NFR-03 security, NFR-04 data integrity, NFR-05 availability, NFR-06 compliance, NFR-07 maintainability, NFR-08 accessibility, NFR-09 auditability, NFR-10 portability. Each carries a verification method, and each is exercised by a test case at Section 9.4 (Functional test cases).

6. Verification

Every requirement in this specification is traceable to at least one test case through the matrix at Section 3.7 (Requirements traceability matrix). Requirements not covered by an executed test are listed as such at Section 9.7 (Requirements verification) rather than presented as satisfied.